

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Домашняя работа 5: изучение основ работы с менеджером
зависимостей prn

Выполнил:

Емельков Матвей

Группа J3210

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

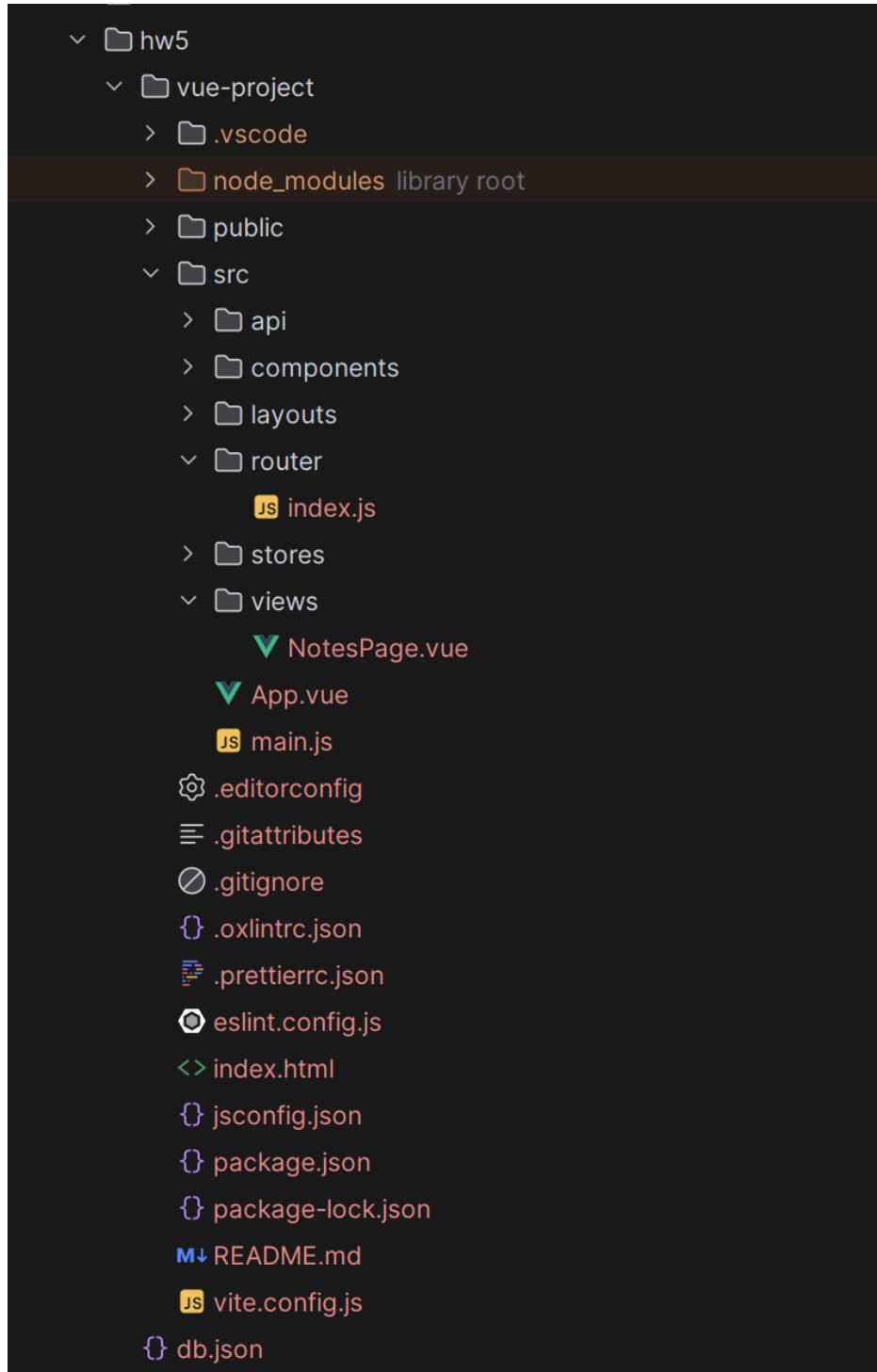
Задача

В рамках данной работы Вам предстоит изучить основные команды пакетного менеджера NPM и научиться стартовать проект на Vue. Научиться работать с npm и vue на основе мануала: <https://docs.google.com/document/d/187UkgGNrcWqkb2aCGpkHTLgeozoElMqdVgVGMBOC9gk/edit?usp=sharing>.

В качестве отчёта ожидаю скриншоты ваших компонентов и код получившегося приложения.

Ход работы

1. С помощью `npm create vue@latest` развернул проект, установил необходимые зависимости (`axios`, `pinia`, `bootstrap`)



2. Настроил axios для взаимодействия с локальным json-server. Для обеспечения модульности логика работы с API вынесена в отдельные файлы (instance.js, [notes.js](#))

```
JS instance.js ×
1 import axios from 'axios'
2
3 const apiURL : string = 'http://localhost:3000'
4 const instance : AxiosInstance = axios.create({baseUrl: apiURL})
5 export default instance Show usages
6
```

```
JS notes.js ×
1 class NotesApi { Show usages
2   constructor(instance) { Show usages
3     this.API = instance
4   }
5
6   getAll = async () : Promise<any> => { Show usages
7     return this.API({url: '/notes'})
8   }
9   createNote = async (data) : Promise<any> => { Show usages
10    return this.API({
11      method: 'POST',
12      url: '/notes',
13      data,
14      headers: {'Content-Type': 'application/json'}
15    })
16  }
17 }
18
19 export default NotesApi Show usages
20
```

3. Настроил хранилище Pinia для управления списком заметок, что позволило реализовать реактивное обновление данных

4. Реализовал компонентную структуру: создал лейаут (BaseLayout.vue) и компонент карточки (NoteCard.vue), а также страница для отображения и создания заметок (NotesPage.vue)

```
✓ BaseLayout.vue ×
1  <template> Show component usages
2    <main class="container my-2"><slot /></main>
3  </template>
4
```

```
✓ BaseLayout.vue  ✓ NoteCard.vue ×
1  <template> Show component usages
2    <div class="card mb-3">
3      <div class="card-body">
4        <h5 class="card-title">{{ name }}</h5>
5        <p class="card-text">{{ text }}</p>
6      </div>
7    </div>
8  </template>
9  <script>
10   export default {props: {name: String, text: String}} Show usages
11 </script>
12
```

NotesPage.js

```
<template>
  <base-layout>
    <h1>Notes app</h1>
    <form @submit.prevent="createCard" class="my-4">
      <input v-model="form.name" class="form-control my-2" placeholder="Название">
      <textarea v-model="form.text" class="form-control my-2" placeholder="Текст"></textarea>
      <button class="btn btn-primary">Отправить</button>
    </form>
    <div v-for="note in notes" :key="note.id">
      <note-card :name="note.name" :text="note.text"/>
    </div>
  </base-layout>
</template>
```

```

<script>
import BaseLayout from '@/layouts/BaseLayout.vue'
import NoteCard from '@/components/NoteCard.vue'
import {mapActions, mapState} from 'pinia'
import useNotesStore from '@/stores/notes'

export default {
  components: {BaseLayout, NoteCard},
  data() {
    return {form: {name: '', text: ''}}
  },
  computed: {...mapState(useNotesStore, ['notes'])},
  methods: {
    ...mapActions(useNotesStore, ['loadNotes',
'createNote']),
    async createCard() {
      await this.createNote(this.form)
      this.form.name = '';
      this.form.text = ''
    }
  },
  mounted() {
    this.loadNotes()
  }
}
</script>

```

5. Настроил двустороннюю привязку данных через v-model и обработка формы через методы Vue, что обеспечило корректную отправку и подгрузку данных без перезагрузки страницы

Notes app

Second Note

some text

Отправить

Тестовая заметка

Привет, это Vue!

First Note

hello



Notes app

Название

Текст

Отправить

Тестовая заметка

Привет, это Vue!

First Note

hello

Second Note

some text



Вывод

В ходе выполнения работы я изучил основы NPM и фреймворка Vue.js. Реализована система взаимодействия фронтенда с API через Pinia и Axios позволила создать интерактивное приложение с реактивным обновлением контента, что значительно упрощает разработку динамических интерфейсов по сравнению с использованием чистого JavaScript